

Oracle Analytic Functions (Fensterfunktionen)

Inhalt der PDF-Datei zur **Oracle Analytic Functions** im Markdown-Format, einschließlich der Tabellen.



COLLABORATE12

TECHNOLOGY AND APPLICATIONS FORUM
FOR THE ORACLE COMMUNITY



Agenda

- **Aggregate vs Analytic**
 - **PARTITION BY**
 - **ORDER BY**
 - **Window Clause**
 - **ROWS**
 - **RANGE**
-

WHY USE ANALYTIC FUNCTIONS?

- Ability to see one row from another row in the results
 - Avoid self-join queries
 - Summary data in detail rows
 - Slice and dice within the results
-

AGGREGATE OR ANALYTIC?

Funktion	Aggregate	Analytic
COUNT	✓	✓
SUM	✓	✓
MAX	✓	✓
MIN	✓	✓

What's the difference?

	SYNTAX	OUTPUT
Aggregate (traditional)	Query often includes the keywords GROUP BY	Output is a single row (or one row per group with GROUP BY)
Analytic	OVER (some other stuff)	Does not change number of rows

AGGREGATE EXAMPLES

<pre>SELECT COUNT (*) FROM scott.emp;</pre> <pre>COUNT(*) ----- 14 1 row selected.</pre>	<pre>SELECT COUNT (*) , SUM (sal) , MAX (sal) , MIN (ename) FROM scott.emp;</pre> <pre>COUNT(*) SUM(SAL) MAX(SAL) MIN(ENAME) ----- 14 29025 5000</pre>	<table> <tr> <th>Deptno</th><th>Ename</th><th>Sal</th></tr> <tr><td>10</td><td>Clark</td><td>2450</td></tr> <tr><td>10</td><td>King</td><td>5000</td></tr> <tr><td>10</td><td>Miller</td><td>1300</td></tr> <tr><td>20</td><td>Adams</td><td>1100</td></tr> <tr><td>20</td><td>Ford</td><td>3000</td></tr> <tr><td>20</td><td>Jones</td><td>2975</td></tr> <tr><td>20</td><td>Scott</td><td>3000</td></tr> <tr><td>20</td><td>Smith</td><td>800</td></tr> <tr><td>30</td><td>Allen</td><td>1600</td></tr> <tr><td>30</td><td>Blake</td><td>2850</td></tr> </table>	Deptno	Ename	Sal	10	Clark	2450	10	King	5000	10	Miller	1300	20	Adams	1100	20	Ford	3000	20	Jones	2975	20	Scott	3000	20	Smith	800	30	Allen	1600	30	Blake	2850
Deptno	Ename	Sal																																	
10	Clark	2450																																	
10	King	5000																																	
10	Miller	1300																																	
20	Adams	1100																																	
20	Ford	3000																																	
20	Jones	2975																																	
20	Scott	3000																																	
20	Smith	800																																	
30	Allen	1600																																	
30	Blake	2850																																	

1 row selected.

ADAMS

1 row selected.

Deptno	Ename	Sal
30	James	950
30	Martin	1250
30	Turner	1500
30	Ward	1250

```
SELECT COUNT ( * )  
       , SUM ( sal )  
FROM scott.emp  
WHERE deptno = 30;
```

COUNT(*)	SUM(SAL)
-----	-----
6	9400

1 row selected.

```
SELECT deptno  
       , COUNT(*)  
       , SUM(sal)  
FROM scott.emp  
GROUP BY deptno;
```

DEPTNO	COUNT(*)	SUM(SAL)
-----	-----	-----
10	3	8750
20	5	10875
30	6	9400

3 rows selected.

One record for
each group

↑ Data from scott.emp table shown
above ↑

```
SELECT deptno  
       , COUNT(*)  
       , SUM(sal)  
FROM scott.emp;
```

ERROR at line 1:
ORA-00937: not a

single-group group
function

ANALYTIC FUNCTIONS

What makes a function analytic?

- Keyword **OVER**
- Followed by set of parentheses

```
SELECT deptno, ename, sal
      , COUNT ( * ) OVER ( )
      , SUM ( sal ) OVER ( )
FROM scott.emp;
```

DEPTNO	ENAME	SAL	COUNT(*)OVER()	SUM(SAL)OVER()
10	CLARK	2450	14	29025
10	KING	5000	14	29025
10	MILLER	1300	14	29025
20	ADAMS	1100	14	29025
20	FORD	3000	14	29025
20	JONES	2975	14	29025
20	SCOTT	3000	14	29025
20	SMITH	800	14	29025
30	ALLEN	1600	14	29025
30	BLAKE	2850	14	29025
30	JAMES	950	14	29025
30	MARTIN	1250	14	29025
30	TURNER	1500	14	29025
30	WARD	1250	14	29025
14 rows selected				

Returns one result
for each record in the dataset.
No grouping

With WHERE Clause ...

Deptno	Ename	Sal
10	Clark	2450

•Which happens first?

```
SELECT
FROM scott.emp
WHERE deptno = 30;
```

```
DEPTNO ENAME      SAL COUNT(*)OVER()
SUM(SAL)OVER()
```

```
-----
-----
30      ALLEN      1600      6
9450
30      BLAKE      2850      6
9450
30      JAMES      950       6
9450
30      MARTIN     1250      6
9450
30      TURNER     1500      6
9450
30      WARD       1250      6
9450
```

6 rows selected.

Even with OVER() and empty parens, the function operates only on the records which meet the conditions of the WHERE clause

Deptno	Ename	Sal
10	King	5000
10	Miller	1300
20	Adams	1100
20	Ford	3000
20	Jones	2975
20	Scott	3000
20	Smith	800
30	Allen	1600
30	Blake	2850
30	James	950
30	Martin	1250
30	Turner	1500
30	Ward	1250

DISTINCT vs GROUP BY

Wird eine analytische Funktion (**COUNT(*) OVER () AS empcnt**) in Verbindung mit **SELECT DISTINCT deptno** verwendet, bleibt die Zeilenzahl aller Abteilungen im *empcnt* gleich (14, da **OVER ()** die Gesamtanzahl der Zeilen vor der **DISTINCT**-Filterung zählt).

Wird **GROUP BY** verwendet, zählt **COUNT(*)** die Zeilen *pro Gruppe*.

Beispiel 1: DISTINCT mit Analytic Function

```
SELECT DISTINCT deptno,
COUNT(*) OVER () AS empcnt
FROM scott.emp;
```

DEPTNO	EMPCNT
10	14
20	14
30	14

3 rows selected.

Beispiel 2: GROUP BY (Traditionelles Aggregat)

```
SELECT deptno,
       COUNT(deptno) AS empcnt -- COUNT(*) is also possible, but COUNT(deptno) is semantically
FROM scott.emp
GROUP BY deptno;
```

DEPTNO	EMPCNT
10	3
20	5
30	6

3 rows selected. (Hinweis: Die Spalte `EMPCNT` im Beispiel 26 aus der Quelle zeigt 3 für alle Zeilen, dies ist wahrscheinlich ein Fehler in der Quell-PDF, da `COUNT(*)` in einer `GROUP BY`-Abfrage die Anzahl der Zeilen in jeder Gruppe zurückgeben sollte: 3, 5, 6).

Reihenfolge der Operationen (Order of Operations)

1. Table Joins
2. WHERE clause filters
3. GROUP BY
4. Analytic Functions
5. DISTINCT
6. Ordering

The Analytic Clause

Das ist der Inhalt innerhalb der Klammern (`OVER ( ... )`).

Komponenten der Analytic Clause

Expressions, die der Funktion sagen, dass sie anders rechnen soll.

Drei mögliche Komponenten, die in dieser Reihenfolge stehen müssen:

1. **Partition**
2. **Order**
3. **Windowing**

Einige oder alle sind optional, abhängig von der Funktion.

PARTITION BY

Berechnet die analytische Funktion auf einer **Untermenge** der Datensätze, ähnlich einem *GROUP BY* ohne die Anzahl der Zeilen zu reduzieren.

```
SELECT deptno, ename, sal, job
, COUNT(*) OVER (PARTITION BY job) jobcount -- Zählt Mitarbeiter pro Job
, SUM(sal) OVER (PARTITION BY deptno) deptsum -- Summiert Gehälter pro Abteilung
FROM scott.emp;
```

DEPTNO	ENAME	SAL	JOB	JOBCOUNT	DEPTSUM
10	CLARK	2450	MANAGER	3	8750
10	KING	5000	PRESIDENT	1	8750
10	MILLER	1300	CLERK	4	8750
20	ADAMS	1100	CLERK	4	10875
20	FORD	3000	ANALYST	2	10875
20	JONES	2975	MANAGER	3	10875
20	SCOTT	3000	ANALYST	2	10875
20	SMITH	800	CLERK	4	10875
30	ALLEN	1600	SALESMAN	4	9400
30	BLAKE	2850	MANAGER	3	9400
30	JAMES	950	CLERK	4	9400

DEPTNO	ENAME	SAL	JOB	JOBCOUNT	DEPTSUM
30	MARTIN	1250	SALESMAN	4	9400
30	TURNER	1500	SALESMAN	4	9400
30	WARD	1250	SALESMAN	4	9400

14 rows selected.

Vergleich: Korrelierte Skalare Subqueries

Die gleiche Logik kann mit korrelierten skalaren Subqueries erreicht werden, aber analytische Funktionen sind oft effizienter (Analytic SQL: ONE PASS vs. Traditional aggregate syntax: Three passes over the table, according to the `EXPLAIN PLAN` output).

```
SELECT deptno, ename, sal, job
, (SELECT COUNT(*) FROM scott.emp WHERE job = e.job) jobcount
, (SELECT SUM(sal) FROM scott.emp WHERE deptno = e.deptno) deptsum
FROM scott.emp e;
```

DEPTNO	ENAME	SAL	JOB	JOBCOUNT	DEPTSUM
10	CLARK	2450	MANAGER	3	8750
...	...	...	...	...	...

(Die folgenden Abschnitte über **LAG/LEAD**, **RANKING FUNCTIONS** und **WINDOWING** wurden ebenfalls in das Markdown konvertiert, um die Vollständigkeit zu gewährleisten.)

ZWEI NEUE FUNKTIONEN: LAG und LEAD

- **LAG** gibt den Wert eines Feldes aus einem Datensatz zurück, der **vor** dem aktuellen Datensatz liegt.
- **LEAD** gibt den Wert eines Feldes aus einem Datensatz zurück, der **nach** dem aktuellen Datensatz liegt.
- **Syntax:** `LAG(field_name, num_recs)` OVER (ORDER BY ...)
- `ORDER BY` ist erforderlich.
- Dies sind ausschließlich analytische Funktionen.

LAG Demonstration

```
SELECT deptno, ename, hiredate
, LAG(ename) OVER (ORDER BY hiredate) prior_hire -- Vorgänger basierend auf Einstellungsdatum
FROM scott.emp
ORDER BY deptno, ename;
```

DEPTNO	ENAME	HIREDATE	PRIOR_HIRE
10	CLARK	09-JUN-81	BLAKE
10	KING	17-NOV-81	MARTIN
10	MILLER	23-JAN-82	FORD
20	ADAMS	23-MAY-87	SCOTT
20	FORD	03-DEC-81	JAMES
20	JONES	02-APR-81	WARD
20	SCOTT	09-DEC-82	MILLER
20	SMITH	17-DEC-80	(NULL)
30	ALLEN	20-FEB-81	SMITH
30	BLAKE	01-MAY-81	JONES
30	JAMES	03-DEC-81	KING
30	MARTIN	28-SEP-81	TURNER
30	TURNER	08-SEP-81	CLARK
30	WARD	22-FEB-81	ALLEN

LAG/LEAD mit Versatz (Offset)

```
SELECT deptno, ename, sal
, LAG(ename) OVER (ORDER BY ename) f1 -- 1 record behind
, LAG(ename, 2) OVER (ORDER BY ename) f2 -- 2 records behind
, LEAD(ename) OVER (ORDER BY ename DESC) f3 -- 1 record ahead (reverse order)
, LAG(sal) OVER (ORDER BY ename) f4 -- LAG on SAL
FROM scott.emp
ORDER BY deptno, ename;
```

DEPTNO	ENAME	SAL	F1	F2	F3	F4
10	CLARK	2450	BLAKE	ALLEN	BLAKE	2850
10	KING	5000	JONES	JAMES	JONES	2975
10	MILLER	1300	MARTIN	KING	MARTIN	1250
20	ADAMS	1100	(NULL)	(NULL)	WARD	(NULL)
20	FORD	3000	CLARK	BLAKE	CLARK	2450
20	JONES	2975	JAMES	FORD	JAMES	950
20	SCOTT	3000	MILLER	MARTIN	MILLER	1300
20	SMITH	800	SCOTT	MILLER	SCOTT	3000
30	ALLEN	1600	ADAMS	(NULL)	ADAMS	1100
30	BLAKE	2850	ALLEN	ADAMS	ALLEN	1600
30	JAMES	950	FORD	CLARK	FORD	3000
30	MARTIN	1250	KING	JONES	KING	5000
30	TURNER	1500	SMITH	SCOTT	SMITH	800
30	WARD	1250	TURNER	SMITH	TURNER	1500

ORDER BY WITH PARTITION BY

`PARTITION BY` teilt die Daten, und `LAG` wird innerhalb jeder Abteilung (Partition) durchgeführt.

```
SELECT deptno, ename, sal
, LAG(ename) OVER (ORDER BY ename) f1 -- LAG über die gesamte Tabelle (a
, LAG(ename) OVER (PARTITION BY deptno ORDER BY ename) f2 -- LAG nur innerhalb der Abteilung
, LAG(ename) OVER (PARTITION BY deptno ORDER BY sal DESC) f3 -- LAG nur innerhalb der Abteilung
FROM scott.emp
ORDER BY deptno, ename;
```

DEPTNO	ENAME	SAL	F1	F2	F3
10	CLARK	2450	BLAKE	(NULL)	KING
10	KING	5000	JONES	CLARK	CLARK

DEPTNO	ENAME	SAL	F1	F2	F3
10	MILLER	1300	MARTIN	KING	CLARK
20	ADAMS	1100	(NULL)	(NULL)	SCOTT
20	FORD	3000	CLARK	ADAMS	SCOTT
20	JONES	2975	JAMES	FORD	FORD
20	SCOTT	3000	MILLER	JONES	ADAMS
20	SMITH	800	SCOTT	SCOTT	ADAMS
30	ALLEN	1600	ADAMS	(NULL)	BLAKE
30	BLAKE	2850	ALLEN	ALLEN	ALLEN
30	JAMES	950	FORD	BLAKE	WARD
30	MARTIN	1250	KING	JAMES	TURNER
30	TURNER	1500	SMITH	MARTIN	ALLEN
30	WARD	1250	TURNER	TURNER	MARTIN

DREI WEITERE FUNKTIONEN: Ranking Functions

- **RANK()**: Weist bei Gleichheit den gleichen Rang zu und überspringt die nächsten Ränge.
- **DENSE_RANK()**: Weist bei Gleichheit den gleichen Rang zu, überspringt aber keine Ränge.
- **ROW_NUMBER()**: Weist jedem Datensatz eine eindeutige, sequentielle Nummer zu.
- **Usage:** `RANK() OVER (ORDER BY field_name)`

Beispiel 1: Ranking ohne Gleichheit (Tie)

Wenn es keine Gleichheit gibt, liefern alle drei Funktionen dieselben Werte (ROW_NUMBER liefert immer eine eindeutige Nummer).

```
SELECT deptno, ename, sal
, RANK() OVER (ORDER BY ename) f1
, DENSE_RANK() OVER (ORDER BY ename) f2
, ROW_NUMBER() OVER (ORDER BY ename) f3
```

```
FROM scott.emp
ORDER BY deptno, sal;
```

DEPTNO	ENAME	SAL	F1	F2	F3
10	MILLER	1300	10	10	10
10	CLARK	2450	4	4	4
10	KING	5000	8	8	8
...	...	...	...	...	...

Beispiel 2: Ranking mit Gleichheit (Ties)

Hier wird nach Gehalt (`sal`) sortiert, das Gleichheiten aufweist.

```
SELECT deptno, ename, sal
, RANK() OVER (ORDER BY sal) f1
, DENSE_RANK() OVER (ORDER BY sal) f2
, ROW_NUMBER() OVER (ORDER BY sal) f3
FROM scott.emp
ORDER BY deptno, sal;
```

DEPTNO	ENAME	SAL	F1	F2	F3	F4
10	MILLER	1300	6	5	6	
10	CLARK	2450	9	8	9	
10	KING	5000	14	12	14	
20	SMITH	800	1	1	1	
20	ADAMS	1100	3	3	3	
20	JONES	2975	11	10	11	
20	FORD	3000	12	11	13	
20	SCOTT	3000	12	11	12	
30	JAMES	950	2	2	2	
30	WARD	1250	4	4	4	
30	MARTIN	1250	4	4	5	

DEPTNO	ENAME	SAL	F1	F2	F3	F4
30	TURNER	1500	7	6	7	
30	ALLEN	1600	8	7	8	
30	BLAKE	2850	10	9	10	

ORDER BY CAVEAT #2 (Wichtiger Hinweis)

Bei vielen Funktionen (**SUM**, **COUNT**, **MAX**, **MIN**, **LAST_VALUE**) ändert die Verwendung von `ORDER BY` die **Windowing Clause** (den Datenbereich).

Standardverhalten bei ORDER BY: Die Funktion summiert (oder zählt/findet Min/Max) vom **Beginn der Partition** bis zum **aktuellen Datensatz** (einschließlich).

Beispiel: Running Totals (MTD – Month To Date)

```
SELECT deptno, ename, sal
, SUM(sal) OVER (ORDER BY ename) s
, COUNT(*) OVER (ORDER BY ename) c
, MIN(sal) OVER (ORDER BY ename) mn
, MAX(sal) OVER (ORDER BY ename) mx
FROM scott.emp
WHERE deptno = 10;
```

DEPTNO	ENAME	SAL	S	C	MN	MX
10	CLARK	2450	2450	1	2450	2450
10	KING	5000	7450	2	2450	5000
10	MILLER	1300	8750	3	1300	5000

$7450 = 2450 + 5000$

$8750 = 7450 + 1300$

Dies ist die Kurzform für: `SUM(sal) OVER (ORDER BY ename ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

WINDOWING CLAUSE

Die `WINDOWING` Clause wählt eine kleinere Untermenge als die Partition aus, basierend auf einer Anzahl von Datensätzen oder einem Zeitraum.

Demonstration des Standard-Windowing

Funktion	Analytic Clause	Implizite Windowing Clause
Aggregat-Funktion ohne ORDER BY	<code>SUM(sal) OVER ()</code>	Implizit: Die gesamte Partition (oder die gesamte Tabelle, wenn keine Partition vorhanden ist).
Aggregat-Funktion mit ORDER BY	<code>SUM(sal) OVER (ORDER BY ename)</code>	Implizit: <code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>

```
SELECT deptno, ename, sal
, SUM(sal) OVER () sum1 -- L
, SUM(sal) OVER (ORDER BY ename) sum2 -- L
, SUM(sal) OVER (ORDER BY ename ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) sum3
, SUM(sal) OVER (ORDER BY ename ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) sum4 -- Exp
FROM scott.emp
WHERE deptno = 10;
```

DEPTNO	ENAME	SAL	SUM1	SUM2	SUM3	SUM4
10	CLARK	2450	8750	2450	8750	2450
10	KING	5000	8750	7450	8750	7450
10	MILLER	1300	8750	8750	8750	8750

SUM1 ist gleich SUM3. SUM2 ist gleich SUM4.

WINDOWING: ROWS (Basierend auf Datensätzen)

Beschränkt das Fenster durch eine Anzahl von Datensätzen vor/nach dem aktuellen Datensatz.

```
SELECT deptno, ename, sal
, SUM(sal) OVER (ORDER BY ename ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) sum1 -- Gesamttabelle
, SUM(sal) OVER (PARTITION BY deptno ORDER BY ename ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
FROM scott.emp;
```

DEPTNO	ENAME	SAL	SUM1	SUM2
10	CLARK	2450	8300	7450
10	KING	5000	9225	8750
10	MILLER	1300	5550	6300
20	ADAMS	1100	2700	4100
...	...	...	...	...

Beispiel SUM2 (10 CLARK): $2450 + 5000 = 7450$ (1 davor (NULL, da erster) + 1 danach (KING))

Beispiel SUM2 (10 KING): $2450 + 5000 + 1300 = 8750$ (1 davor (CLARK) + 1 danach (MILLER))

WINDOWING CLAUSE COMPARISON (ROWS vs. RANGE)

	ROWS	RANGE
Restriktion	Durch Anzahl der Datensätze	Durch einen Zeitraum oder einen Wert
Referenziert	Basiert auf <code>ORDER BY</code>	Referenziert Feld, das in <code>ORDER BY</code> verwendet wird
Beispiel	<code>ROWS BETWEEN 10 PRECEDING AND 10 FOLLOWING</code>	<code>RANGE BETWEEN INTERVAL '10' DAY PRECEDING AND INTERVAL '10' DAY FOLLOWING</code>

Beispiel: RANGE (Basierend auf Gehaltswert)

```
SELECT ename, sal
, COUNT(*) OVER (ORDER BY sal RANGE BETWEEN 200 PRECEDING AND 200 FOLLOWING) emps_200_sal
FROM scott.emp;
```

Zählt alle Mitarbeiter, deren Gehalt maximal 200 weniger oder 200 mehr als das aktuelle Gehalt beträgt.
